

Veridex Production Readiness Roadmap

A six-phase engineering plan that takes Veridex from a working scaffold to a production-grade, enterprise-ready AI governance platform — engineered so that speed and smoothness are the product's defining experience, not an afterthought.

DOCUMENT TYPE

Phased Architecture & Delivery Plan

VERSION

v1.0 — July 2026

every interaction feel instant, never lagging

03

Treat governance and security as first-class UX

04

Ship AI features that feel alive, not bolted-on

Table of Contents

A phase-by-phase build order — each phase unlocks the next

—	Executive Summary	Current state, the smoothness thesis, and how to read this roadmap	03
00	Phase 0 — Stabilize the Foundation	Fix auth, tenant headers, env config, and the typed API client	05
01	Phase 1 — Governance & Access Control	Policy engine, RBAC seeding, audit trail, zero-lockout guarantees	08
02	Phase 2 — The Smoothness Layer	Global error handling, optimistic UI, skeletons, resilience patterns	11
03	Phase 3 — AI & Knowledge Core	RAG pipeline, knowledge graph, agent runtime, streaming chat	14
04	Phase 4 — Observability & Resilience	Tracing, dashboards, circuit breakers, self-healing connectors	17
05	Phase 5 — Production Hardening & Launch	Testing, CI/CD, performance budgets, accessibility, launch readiness	20
—	Cross-Cutting: UI/UX Design System	Motion, feedback, latency budgets, and the design language	23
—	Roadmap Summary & Sequencing	Timeline view, dependencies, and recommended team allocation	25

Where Veridex Stands Today

Veridex already has the hard infrastructure decisions made correctly: Postgres, Redis, Qdrant, Neo4j, MinIO, and full OpenTelemetry tracing are wired into a FastAPI backend with async SQLAlchemy, and the Next.js 16 frontend is scaffolded with a typed API client, React Hook Form, and shadcn/ui. This is a strong skeleton. What it lacks is **connective tissue** — the layer of work that turns a correct architecture into a product that feels fast, safe, and trustworthy under real use.

This report treats that connective tissue as the actual work of the next phase, not a cleanup task tacked onto feature development. Every phase below is scoped around one question: *does this make Veridex more stable, or does it just make it bigger?* Stability, security, and speed come first because every AI feature, every connector, and every dashboard built on top of a shaky auth layer or an undefined policy engine will need to be rebuilt later.

THE SMOOTHNESS THESIS

Smoothness is not a UI theme — it's an architectural property. It comes from four disciplines applied consistently everywhere: (1) **never blocking the main thread** on a network call, (2) **always showing the user a next state** — a skeleton, an optimistic update, a progress indicator — never a blank screen, (3) **failing gracefully and specifically**, so a 403 never looks like a 500 and a timeout never looks like a crash, and (4) **keeping perceived latency under ~150ms** for any local interaction and under ~1s for any network round trip, with a busy-state shown past 100ms. Phase 2 of this roadmap operationalizes this as a reusable layer rather than a per-page habit.

How to Read This Roadmap

Each phase has a fixed shape: a **goal**, a **backend workstream**, a **frontend workstream**, a **smoothness/UX commitment** specific to that phase, a **risk callout**, and an **exit criteria checklist**. A phase is not "done" by calendar time — it's done when its exit criteria are met. Phases are ordered so each one removes a blocker for the next: you cannot build reliable AI chat (Phase 3) on top of an undefined policy engine (Phase 1), and you cannot claim production-readiness (Phase 5) without observability (Phase 4) already in place to prove it.



Guiding Principles Applied Throughout

- **Fix before you extend.** Phases 0–1 touch no new features — they make existing ones correct.
- **Type safety end-to-end.** The OpenAPI-generated client is the only way the frontend talks to the backend after Phase 0.
- **Deny-by-default is a UX problem too.** A policy engine that silently locks out users is a smoothness failure, not just a security one.
- **Every async action gets a visible state.** Loading, success, error, and empty states are mandatory, not optional polish.
- **Observability precedes scale.** You cannot make something smooth that you cannot measure.
- **New features inherit the design system.** Chat, agents, and knowledge graph UIs reuse Phase 2's primitives — never bespoke patterns.

Phase 0 — Stabilize the Foundation

Close every gap identified in the current architecture report before any new feature work begins. This phase produces zero user-facing features and is the highest-leverage work in the entire roadmap — everything downstream depends on it being done correctly.

- Blocking Priority
- No New Surface Area
- Backend + Frontend

Backend Workstream

Typed API Client as the Single Source of Truth

Run `npm run generate-client` against the live `openapi.json` and commit the generated client. Every existing native `fetch` call — starting with the login page — is replaced with the typed client. This is not cosmetic: a typed client turns backend contract changes into compile-time frontend errors instead of silent runtime failures, which is the single biggest lever for reducing production incidents in a fast-moving API.

Multi-Tenant Header Propagation

The backend already expects `X-Tenant-ID` on every protected request via `OPAMiddleware`. Add a single Axios/fetch interceptor on the frontend that reads the active tenant from `AuthContext` and injects the header automatically. No component should ever manually set this header — centralizing it here prevents an entire class of "works on my tenant" bugs.

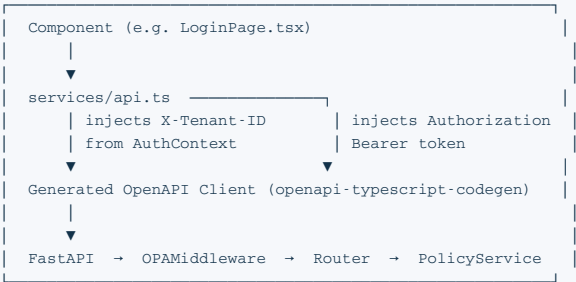
Environment & OAuth Configuration

Populate `GITHUB_CLIENT_ID`, `GITHUB_CLIENT_SECRET`, `GOOGLE_CLIENT_ID`, and `GOOGLE_CLIENT_SECRET` in `.env`, register both OAuth apps with correct callback URLs, and add a startup-time environment validator (e.g. Pydantic `BaseSettings` with required fields) so the API refuses to boot with a misconfigured environment rather than failing unpredictably on first OAuth attempt.

DELIVERABLE	PRIORITY	DEFINITION OF DONE
Typed client wired into login/register	P0	Native <code>fetch</code> calls removed from <code>/login</code> , <code>/register</code> , <code>/mfa</code>
Tenant header interceptor	P0	Verified via network inspector on every authenticated request
OAuth env vars + startup validation	P0	App fails fast with a clear message if any required var is missing
Pydantic <code>Settings</code> class centralizing config	P1	No <code>os.environ.get()</code> calls scattered across routers
API service wrapper module (frontend)	P0	Single import path for all API calls; interceptors live here

Frontend Workstream

Introduce a centralized `services/api.ts` that wraps the generated OpenAPI client with three concerns baked in: tenant header injection, auth token attachment, and a hook for the global error interceptor that Phase 2 will complete. Every page — even the ones currently working — is migrated to import from this module rather than calling `fetch` directly. This is deliberately done in Phase 0, not deferred, because retrofitting it after ten more pages exist is dramatically more expensive.



SMOOTHNESS COMMITMENT — PHASE 0

Even in a phase with no visible features, the user-facing effect matters: OAuth buttons that are configured but silently fail are worse than no OAuth buttons at all. Every auth pathway either works end-to-end or is hidden from the UI behind a feature flag until it does. Nothing half-works in front of a user.

RISK — MIGRATION REGRESSIONS

Replacing working `fetch` calls with a typed client can introduce subtle payload-shape mismatches. Mitigate by migrating one route at a time behind manual QA, not as a single big-bang PR, and by snapshotting current request/response payloads before migrating.

EXIT CRITERIA

- Zero native `fetch` calls remain in the frontend codebase
- `X-Tenant-ID` is present on 100% of authenticated network requests, verified in staging

- ✓ OAuth login succeeds end-to-end for both GitHub and Google in staging
- ✓ API boots and fails clearly (not silently) on missing required environment variables

Phase 1 — Governance & Access Control

Flesh out the Policy Engine so that "deny by default" never means "lock everyone out." This phase turns OPAMiddleware from a liability into the platform's trust anchor, and gives admins a real interface to grant, audit, and revoke access.

- Security Critical
- Unlocks All Protected Routes
- Backend + Frontend

Backend Workstream

PolicyService Implementation

Implement `PolicyService.evaluate()` against a concrete role-permission model: roles (`owner` , `admin` , `member` , `viewer`) mapped to resource-action pairs (e.g. `projects:write` , `governance:read`). Store role bindings per-tenant in Postgres, cache resolved permissions in Redis with a short TTL, and invalidate the cache on any role change so permission edits take effect within seconds, not minutes.

Default Role Seeding & Migration Safety

Add an Alembic migration that seeds an `owner` role for the first registered user of every new tenant, so no tenant can ever exist in a state where zero users have access. This single guarantee eliminates the most dangerous failure mode of a deny-by-default system: a correctly-configured backend that nobody can log into.

Governance API

Build out `/governance` to support: listing roles and their permissions, assigning/revoking roles per user, viewing the immutable audit log with filters (actor, resource, time range, decision), and a policy simulation endpoint that answers "would this user be allowed to do X?" without actually performing the action — critical for both admin UI and automated testing.

DELIVERABLE	PRIORITY	DEFINITION OF DONE
<code>PolicyService.evaluate()</code> logic	P0	Resolves role → permission → decision for any resource/action pair
Owner role auto-seeding on tenant creation	P0	New tenants never start in a fully-locked state
Redis-cached permission resolution	P1	Permission checks add <5ms p95 to request latency
<code>/governance/roles</code> CRUD endpoints	P0	Admins can create/edit/assign roles via API
<code>/governance/audit-log</code> query endpoint	P1	Filterable, paginated, backed by <code>ImmutableAuditLog</code>
Policy simulation endpoint	P2	Dry-run permission checks for UI and tests

Frontend Workstream

Build out the `/policies` shell into a real governance console: a role matrix (roles as rows, permissions as columns) that's editable inline, a searchable/filterable audit log table, and a "test as user" panel that calls the policy simulation endpoint so admins can debug access issues without impersonation. Every permission change shows an immediate optimistic update, confirmed against the Redis-invalidated cache within the same request cycle.

Role Matrix UI

A dense, spreadsheet-like grid rather than a form-per-permission. Toggling a cell fires a single PATCH, shows a subtle checkmark pulse on success, and reverts with an inline error badge on failure — no modal, no page reload.

Audit Log Explorer

Virtualized table (handles 100k+ rows without jank), sticky filter bar, and a detail drawer that slides in from the right on row click rather than navigating away from the filtered list.

SMOOTHNESS COMMITMENT — PHASE 1

A 403 must never feel like a dead end. Every permission-denied response includes a machine-readable reason code that the frontend maps to a specific, actionable message — "Ask an admin to grant you `projects:write`" rather than a generic "Forbidden." Where possible, the denied action's button is disabled proactively based on cached permissions, so the user never clicks into a wall in the first place.

RISK — CACHE STALENESS ON PERMISSION REVOCATION

A cached "allow" decision that outlives a revoked permission is a security gap, not just a UX bug. Mitigate with short TTLs (≤30s) plus explicit cache invalidation on every role-mutation endpoint, and add an integration test that revokes a permission and asserts the next request is denied within the TTL window.

EXIT CRITERIA

- ✓ No user can be created into a tenant with zero effective permissions
- ✓ Role changes propagate to enforcement within 30 seconds, verified by automated test
- ✓ Every 403 in the UI shows a specific, actionable reason — never a bare "Forbidden"
- ✓ Audit log is queryable by actor, resource, and time range from the governance console

Phase 2 — The Smoothness Layer

Build the reusable infrastructure that makes every future page feel instant and trustworthy by default: global error handling, token refresh, optimistic UI primitives, and a consistent loading/empty/error state system. This is the phase that directly delivers on smoothness as the platform's USP.

USP Delivery Phase

Reusable Primitives

Primarily Frontend

Frontend Workstream

Global Interceptor & Silent Token Refresh

Extend the Phase 0 API wrapper with a response interceptor that catches `401`, silently calls `/auth/refresh`, retries the original request once, and only redirects to `/login` if refresh itself fails. Done correctly, a user's access token can expire mid-session and they will never notice — no forced re-login, no lost form state, no interrupted chat stream.

Unified Toast & Notification System

Replace ad-hoc component-level error strings with a single `shadcn / sonner`-based toast system fed by the global interceptor. Errors are categorized (network, validation, permission, server) and each category has a distinct visual treatment and default message, with room for a component to override the message when it has more context.

Skeleton, Optimistic Update & Empty-State Kit

Ship a small library of primitives — `<Skeleton />` variants matching real content shapes, an `useOptimisticMutation()` hook that updates local state immediately and reconciles or rolls back on server response, and a standard `<EmptyState />` component — so every new page (chat, connectors, knowledge graph) gets consistent, tested loading behavior for free instead of reinventing it.

DELIVERABLE	PRIORITY	DEFINITION OF DONE
Silent token refresh interceptor	P0	Expired token mid-session never forces a visible re-login
Global toast/error system	P0	All API errors route through one component; zero raw error strings in UI
<code>useOptimisticMutation()</code> hook	P0	Used for all create/update/delete actions app-wide
Skeleton component library	P1	One skeleton variant per major content shape (list, card, table, chat)
Route-level suspense boundaries	P1	Every protected route shows a skeleton, never a blank frame, on navigation
Offline/reconnect detection banner	P2	Non-blocking banner on connection loss; auto-dismisses on reconnect

Backend Workstream

Smoothness is enforced from the server side too. Standardize every error response into one envelope shape — `{ "code": "PERMISSION_DENIED", "message": "...", "detail": {...} }` — via a global FastAPI exception handler, so the frontend interceptor can categorize errors reliably without string-matching. Add response-time budgets to `RequestContextMiddleware`: any endpoint exceeding 500ms logs a warning with the trace ID, feeding directly into Phase 4's observability work.

```
// Standard error envelope — every 4xx/5xx returns this shape
{
  "code": "PERMISSION_DENIED",
  "message": "You need 'projects:write' to perform this action.",
  "detail": { "resource": "projects", "action": "write", "tenant_id": "..."},
  "request_id": "req_8f2a1c..."
}
```

SMOOTHNESS COMMITMENT — PHASE 2 (THIS PHASE IS THE COMMITMENT)

Three numeric budgets become non-negotiable engineering constraints from this phase forward: local interactions (toggles, tab switches, form edits) render in **under 100ms**; any action requiring a network round trip shows a busy indicator if it exceeds **100ms** and a skeleton if it exceeds **300ms**; and no navigation between authenticated pages ever shows a fully blank white frame, even on a cold cache. These budgets are checked in Phase 5's performance testing, but they are built here.

RISK — OPTIMISTIC UPDATES MASKING REAL FAILURES

Optimistic UI that never clearly reconciles a rollback can leave users trusting a state the server rejected. Every `useOptimisticMutation()` rollback must pair with a visible toast explaining what reverted and why — silence on rollback is worse than no optimism at all.

EXIT CRITERIA

- ✓ A simulated expired token never produces a visible login redirect during an active session
- ✓ Every mutating action in the app (create/update/delete) uses the optimistic mutation hook
- ✓ No blank-screen navigation exists between any two authenticated routes

✓ 100% of API error responses conform to the standard envelope shape

Phase 3 — AI & Knowledge Core

Build the connective tissue for the AI-native parts of the product: real RAG over Qdrant, graph traversal over Neo4j, and a conversational agent runtime for the /chat and /agents routers. Every AI interaction inherits Phase 2's smoothness primitives — streaming, not spinning; partial results, not all-or-nothing loads.

- New Capability
- Depends on Phases 0-2
- Backend-Heavy

Backend Workstream

RAG Pipeline (Qdrant)

Implement an ingestion service that chunks uploaded documents (from MinIO), generates embeddings, and upserts into Qdrant with tenant-scoped collections so no query can ever leak across tenants. Build the retrieval side as a reusable `RetrievalService` with configurable top-k, similarity threshold, and re-ranking, callable from both `/knowledge` and `/chat`.

Knowledge Graph (Neo4j)

Define an entity/relationship schema for the domains Veridex governs (projects, policies, connectors, agents) and implement a Cypher query service with parameterized templates — never raw string-built Cypher — to eliminate injection risk. Expose a `/knowledge/graph` endpoint returning a normalized node/edge shape the frontend graph visualizer can render directly.

Agent Runtime & Streaming Chat

Implement the `/chat` conversational loop using Server-Sent Events so tokens stream to the client as they're generated, with conversational memory persisted per-session in Redis and long-term summaries persisted in Postgres. The `/agents` router gets a lifecycle state machine (`idle` → `planning` → `executing` → `awaiting_approval` → `complete/failed`) with every transition written to the audit log from Phase 1.

Deliverable	Priority	Definition of Done
Tenant-scoped Qdrant ingestion pipeline	P0	Cross-tenant retrieval is architecturally impossible, not just filtered
<code>RetrievalService</code> with re-ranking	P1	Shared by chat and knowledge endpoints; unit tested for relevance
Neo4j schema + parameterized Cypher service	P0	Zero raw string-concatenated queries in codebase
SSE streaming for <code>/chat</code>	P0	First token appears in <800ms p95; full response streams token-by-token
Agent lifecycle state machine	P1	All transitions audited; stuck agents auto-timeout after a configurable window
Gemini prompt templates + guardrails	P1	Versioned prompts, tenant-configurable system instructions

Frontend Workstream

Build the chat interface around streaming from day one — tokens render as they arrive with a subtle cursor, not a spinner-then-dump. The knowledge graph page uses a force-directed layout (client-side rendering, virtualized for graphs over ~500 nodes) with click-to-expand rather than rendering the entire graph at once. The agents page shows live status via the same SSE channel used for chat, so a running agent's progress is visible without polling.

Chat Experience

Streaming response text, inline source citations from RAG retrieval shown as expandable chips, and a stop-generation control that actually cancels the SSE stream server-side rather than just hiding the UI.

Knowledge Graph Explorer

Pan/zoom canvas, node search with fly-to-node animation, and a side panel showing entity detail on selection — the graph never re-fetches on every interaction, only on explicit expand.

Smoothness Commitment — Phase 3

AI latency is inherently variable, so the product must set honest expectations rather than hide the wait. Streaming beats spinners because it turns "waiting" into "reading" — the moment the first token arrives, perceived latency effectively drops to near-zero even if total generation takes ten seconds. Every AI-driven action shows what stage it's in (retrieving, thinking, generating) rather than a single ambiguous loading state.

Risk — Unbounded LLM Cost or Runaway Agents

A conversational loop or agent runtime without limits can silently run expensive Gemini calls in a loop. Mitigate with a hard per-session token budget, a max-turn count on agent execution, and Redis-backed rate limiting on `/chat` and `/agents` reusing the existing `SlowAPI` infrastructure.

Exit Criteria

- ✓ Chat responses stream token-by-token with first-token latency under 800ms p95
- ✓ RAG retrieval is verifiably tenant-isolated under a cross-tenant penetration test
- ✓ Knowledge graph renders and remains interactive at 1,000+ nodes without frame drops
- ✓ Every agent run has a bounded max duration and appears in the audit log

Phase 4 — Observability & Resilience

Turn the already-provisioned Jaeger, Prometheus, and Grafana stack into something the team actually uses daily, and give the platform the ability to detect and recover from partial failures — a slow connector or a flaky LLM call should degrade gracefully, never cascade.

- Reliability Critical
- Enables Phase 5
- Backend + Infra

Backend Workstream

Distributed Tracing Completion

Propagate OpenTelemetry trace context through every async boundary — Redis cache calls, Qdrant/Neo4j queries, and outbound Gemini API calls — so a single Jaeger trace shows the full request lifecycle, including AI latency broken down by retrieval vs. generation. Currently-skeletal `/observability` and `/telemetry` routers get real handlers exposing trace search and service health to the frontend.

Circuit Breakers & the /resilience Router

Implement circuit breakers (open/half-open/closed) around every external dependency — Gemini API, OAuth providers, third-party connectors — configurable per-tenant through `/resilience`. When a breaker opens, dependent endpoints return a clear degraded-mode response instead of hanging or 500ing, and the frontend already knows (via Phase 2's error envelope) how to render that state distinctly.

Grafana Dashboards & Alerting

Ship pre-built Grafana dashboards for: API latency percentiles by route, error rate by tenant, LLM token usage/cost by tenant, and OPA policy decision volume. Wire Prometheus alerting rules for p95 latency breaches, elevated 5xx rates, and circuit breakers stuck open — routed to a real notification channel, not just a dashboard nobody watches.

DELIVERABLE	PRIORITY	DEFINITION OF DONE
End-to-end trace propagation	P0	Single Jaeger trace covers HTTP → DB → vector store → LLM call
Circuit breakers on all external deps	P0	Simulated Gemini outage degrades gracefully, doesn't cascade to 500s
4 production Grafana dashboards	P1	Latency, errors, LLM cost, policy decisions — all tenant-filterable
Prometheus alert rules + notification routing	P1	Alerts reach Slack/email within 60s of threshold breach
<code>/observability</code> and <code>/telemetry</code> real handlers	P2	Frontend can query service health without direct Grafana access

Frontend Workstream

Build out the `/telemetry` and `/resilience` shells into an internal status console: a service health grid (green/yellow/red per dependency), a live circuit-breaker state panel with manual override for admins, and an embedded or linked view into the relevant Grafana dashboard filtered to the current tenant. This is an internal/admin-facing surface, so it favors information density over the consumer-grade polish of the rest of the app — but still inherits Phase 2's loading and error primitives.

SMOOTHNESS COMMITMENT — PHASE 4

Resilience is smoothness under stress. A user should be able to keep working — browsing projects, reading past chat history, editing drafts — even while the LLM provider is degraded or a connector is down. Degraded mode is a defined, designed state in every major feature, not an unhandled exception path.

RISK — ALERT FATIGUE

Over-broad alerting thresholds trained the team to ignore alerts within the first month at most organizations that skip this step. Start with a small number of high-signal alerts (p95 latency, 5xx rate, breaker state) and tune thresholds against two weeks of real staging traffic before enabling paging for any of them.

EXIT CRITERIA

- ✓ A single trace ID reconstructs the full path of any request across all services
- ✓ A simulated Gemini API outage does not produce any 500 errors — only degraded responses
- ✓ All four core Grafana dashboards are live and tenant-filterable
- ✓ On-call engineer can diagnose a production incident using only dashboards + traces, no code reading

Phase 5 — Production Hardening & Launch

Close the loop with automated testing, CI/CD, formal performance and accessibility budgets, and a launch checklist. This phase proves — with evidence, not opinion — that every commitment made in Phases 0 through 4 actually holds under real conditions.

- Launch Gate
- Depends on All Prior Phases
- Backend + Frontend + Infra

Testing & CI/CD

Backend: pytest + Test Database Isolation

Every router gets contract tests against a real (containerized, ephemeral) Postgres instance — not mocks — including explicit tests for the OPA middleware's deny-by-default behavior, tenant isolation in RAG retrieval, and circuit breaker state transitions. Target 80%+ coverage on `services/`, with 100% coverage on `PolicyService` and `RetrievalService` specifically, since those are the highest-blast-radius modules.

Frontend: Jest + React Testing Library + Playwright

Unit tests for the Phase 2 primitives (optimistic mutation hook, error interceptor, skeleton components) since they're reused everywhere and a regression there breaks the whole app. Playwright end-to-end tests cover the critical paths: register → login → MFA → first project created, and a full chat conversation with streaming assertions.

CI/CD Pipeline

GitHub Actions pipeline: lint → typecheck → unit tests → build → Playwright E2E against a preview deployment → deploy. No merge to main without a green pipeline. Migrations run automatically against a shadow database in CI to catch Alembic migration errors before they hit staging.

Deliverable	Priority	Definition of Done
pytest suite with ephemeral test DB	P0	80%+ coverage on services/, 100% on PolicyService & RetrievalService
Jest + RTL suite for shared components	P0	All Phase 2 primitives have unit tests
Playwright E2E for critical paths	P0	Auth flow, project creation, and chat streaming covered
GitHub Actions CI/CD pipeline	P0	No merge to main without green pipeline; auto-deploy to staging
Performance budget enforcement in CI	P1	Lighthouse CI fails the build below defined thresholds
Accessibility audit (WCAG 2.1 AA)	P1	axe-core scan passes on all major pages

Performance & Accessibility Budgets

These are the numeric proof points for the smoothness thesis stated in the Executive Summary. Each is enforced automatically in CI, not checked manually before launch.

Metric	Budget	Why it matters
Largest Contentful Paint (LCP)	< 2.0s	First meaningful content the user actually came for
Interaction to Next Paint (INP)	< 150ms	Every click/tap feels immediate, not laggy
Cumulative Layout Shift (CLS)	< 0.05	Skeletons match real content dimensions exactly — nothing jumps
API p95 latency (non-AI routes)	< 300ms	Governance, projects, connectors feel instant
Chat first-token latency	< 800ms p95	Streaming starts before the user's attention drifts
Time to Interactive on cold load	< 3.0s	New users don't bounce before the app is usable

Launch Readiness Checklist

- ✓

All Phase 0-4 exit criteria met and re-verified in staging
- ✓

Load test at 3x expected launch traffic with no error rate increase
- ✓

Secrets rotated out of any development defaults
- ✓

Database backup and restore drill completed successfully
- ✓

Incident response runbook written and on-call rotation staffed
- ✓

Rollback procedure tested against the CI/CD pipeline
- ✓

Legal/compliance review of audit log retention policy
- ✓

Status page live and connected to Prometheus alerting

Smoothness Commitment — Phase 5

This phase doesn't add new smoothness — it proves the smoothness built in Phases 2 through 4 survives contact with real users, real load, and real failure modes. A feature that's smooth in a demo but untested under load isn't smooth, it's lucky. Every budget in the table above is a promise made concrete and enforced by machine, not memory.

EXIT CRITERIA — LAUNCH GATE

- ✔ All performance and accessibility budgets pass in CI on the release candidate
- ✔ Zero P0/P1 bugs open against any prior phase's exit criteria
- ✔ Load test and backup/restore drill both completed with signed-off results
- ✔ On-call rotation, runbook, and status page are live before the first external user

UI/UX Design System — The Language of Smoothness

This section is not a phase — it's a set of standards that apply from Phase 2 onward and every page built afterward must comply with. Treat this as the style guide that keeps six phases of work, potentially built by different people, feeling like one coherent product.

Motion & Feedback Rules

- **Duration:** micro-interactions (hover, toggle) at 100–150ms; page-level transitions at 200–300ms. Nothing animates longer than 400ms — motion should never make the user wait.
- **Easing:** ease-out for elements entering the screen, ease-in for elements leaving. Linear easing is reserved for progress indicators only.
- **Every click gets a reaction within one frame** — a pressed state, a ripple, a disabled cursor — even before the network call resolves.
- **Reduced motion is respected.** All animation respects `prefers-reduced-motion` and degrades to instant state changes.

State Coverage Requirement

Every view that fetches data must explicitly design for five states, not just the happy path: **loading** (skeleton matching real layout), **empty** (explains why it's empty and what to do next, never just blank), **error** (specific, actionable, via the Phase 2 toast system), **partial** (some data loaded, some failed — show what succeeded), and **success**. A component review that only screenshots the success state is not considered complete.

Visual Language

Color as Signal, Not Decoration

The deep navy/cyan palette used throughout this document reflects the intended product palette: navy for structure and chrome, cyan as the single accent reserved for primary actions and active states, amber for warnings, red reserved exclusively for destructive/error states so it retains urgency. No other accent colors compete for attention.

Typography & Density

Consumer-facing pages (chat, projects, dashboards) favor generous whitespace and larger type for scanability. Admin/internal pages (governance, telemetry, resilience) favor information density — smaller type, tighter tables — because their users are power users optimizing for throughput, not first impressions.

Accessibility as a Default, Not an Add-On

- All interactive elements reachable and operable via keyboard alone, with a visible focus ring that meets contrast requirements.
- Color is never the sole carrier of meaning — status always pairs an icon or label with its color.
- All form inputs have associated labels; all async state changes are announced to screen readers via ARIA live regions.

Sequencing & Timeline

The phases below are sequential by dependency, not strictly by calendar — Phase 3 (AI Core) can begin its backend groundwork in parallel with the tail end of Phase 2 if team size allows, since it depends on Phase 1's governance layer more than Phase 2's frontend polish. Everything else should not be parallelized: Phases 0 and 1 in particular are correctness-critical and rushing them creates rework that costs more time later than it saves now.

PHASE	FOCUS	DURATION	HARD DEPENDENCY	PRIMARY OUTPUT
0	Stabilize the Foundation	Weeks 1-2	None	Typed client, tenant headers, working OAuth
1	Governance & Access Control	Weeks 3-5	Phase 0	Working policy engine, no-lockout guarantee
2	The Smoothness Layer	Weeks 6-8	Phase 0	Global error handling, optimistic UI kit
3	AI & Knowledge Core	Weeks 9-13	Phases 1 & 2	Streaming chat, RAG, knowledge graph
4	Observability & Resilience	Weeks 14-16	Phase 3	Full tracing, circuit breakers, dashboards
5	Production Hardening & Launch	Weeks 17-20	All prior	Test coverage, CI/CD, launch sign-off

Recommended Team Allocation

If team size allows parallel tracks: one backend-leaning engineer can own Phase 1's PolicyService while a frontend-leaning engineer begins Phase 2's primitives once Phase 0's API wrapper lands — both converge before Phase 3 starts, since Phase 3 needs both a working policy layer and a working error/loading system to build on safely.	If working solo or with a small team: follow the phases strictly in order. The temptation to jump to Phase 3 (the most visible, "impressive" phase) before Phases 0-2 are solid is the single most common way this kind of roadmap fails — the AI features end up sitting on an unstable, unsmooth foundation and every subsequent bug costs more to fix.
---	--

Closing Note

The single sentence that should govern every trade-off decision across all six phases is this: **a feature that works but feels slow, uncertain, or unsafe is not actually done.** Veridex's technical foundations are strong enough to support a genuinely fast, trustworthy product — this roadmap is the plan for making sure the experience matches that foundation at every layer, from the policy engine to the pixel.